



SDI – Sistemas Distribuidos e Internet

ENUNCIADO PRÁCTICA 2 – NODE.JS Y SERVICIOS WEB

INFORME

Nombre/Apellidos, email de UNIOV, I ID-GIT#1	Omayma El Imami Gaada , UO300838@uniovi.es , 2526-703
Nombre/Apellidos, email de UNIOV, ID-GIT#2	Ikram El Mabrouk Morhnane , UO301848@uniovi.es , 2526-704
Nombre/Apellidos, email de UNIOV, ID-GIT#3	Moisés Montes Iglesias, uo296102@uniovi.es , 2526-710
Nombre/Apellidos, email de UNIOV, ID-GIT#4	Mario de la Mora Martínez , UO281860@uniovi.es ,2526-711
Cód. ID EQUIPO	72
URL Repositorio GitHub	https://github.com/Ikram-em/sdi2526-entrega2-72.git

Índice



1	INTRODUCCIÓN	2
2	DIAGRAMA DE NAVEGABILIDAD	3
3	DIAGRAMA ENTIDAD-RELACIÓN	5
4	ASPECTOS TÉCNICOS Y DE DISEÑO RELEVANTES	6
5	DESCRIPCIÓN DE FUNCIONALIDADES DESARROLLAS POR CADA MIEMBRO DEL EQUIPO	7
6	CONCLUSIÓN INDIVIDUAL DE CADA MIEMBRO DEL GRUPO.	13
7	INFORMACIÓN NECESARIA PARA EL DESPLIEGUE Y EJECUCIÓN.....	14
8	CONCLUSIÓN	14

1 Introducción

Este proyecto desarrolla una aplicación de gestión de reservas de espacios para una organización. El sistema contempla tres tipos de usuario:

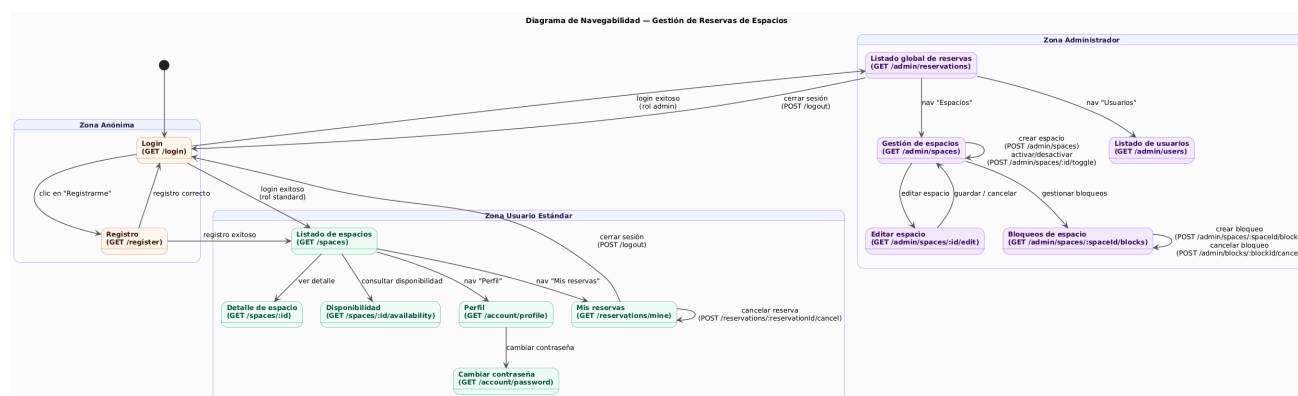
- Usuario anónimo
- Usuario estándar
- Usuario administrador

Cada uno de ellos dispone de un conjunto concreto de funcionalidades y permisos.

La aplicación web permite el registro de nuevos usuarios, el inicio y cierre de sesión, la consulta de espacios disponibles, la visualización de su detalle, la comprobación de disponibilidad en un rango temporal, la consulta y cancelación de reservas propias y el cambio de contraseña. En el caso del administrador, además, se incluyen funcionalidades de gestión de espacios, creación y cancelación de bloqueos, visualización global de reservas, listado de usuarios y exportación de información en formato CSV.

A nivel técnico, la solución se apoya en Node.js, Express, EJS, MongoDB y Mongoose. Junto a la parte web, el repositorio incluye también una API REST con autenticación mediante token para la gestión de reservas y un cliente React preparado para consumir dicha API.

2 Diagrama de navegabilidad



En primer lugar, el usuario anónimo puede acceder únicamente a las pantallas de inicio de sesión y registro, disponibles en las rutas **/login** y **/register**. Una vez completado el registro correctamente, el usuario es redirigido al listado de espacios (**/spaces**). En caso de iniciar sesión, la redirección depende del rol asociado a la cuenta.

El usuario estándar accede a la parte funcional principal de la aplicación. Desde el listado de espacios (**/spaces**) puede consultar el detalle de cada espacio (**/spaces/:spaceId**) y su disponibilidad (**/spaces/:spaceId/availability**). Además, dispone de una sección para consultar sus propias reservas (**/reservations/mine**) y de una vista para modificar su contraseña (**/account/password**), así como una vista de perfil (**/account/profile**).

El usuario administrador dispone de un flujo de navegación orientado a tareas de gestión y supervisión. Puede consultar el listado global de reservas (**/admin/reservations**), gestionar los espacios (**/admin/spaces**), editar un espacio concreto (**/admin/spaces/:spaceId/edit**), gestionar los bloqueos asociados a cada espacio (**/admin/spaces/:spaceId/blocks**) y consultar el listado de usuarios registrados (**/admin/users**).

Existen además varias acciones protegidas que requieren autenticación y permisos adecuados. Entre ellas se incluyen el cierre de sesión (**POST /logout**), la cancelación de reservas propias (**POST /reservations/:reservationId/cancel**), la creación, edición y activación/desactivación de espacios (**/admin/spaces** y **/admin/spaces/:spaceId/toggle**), la creación y cancelación de bloqueos (**POST**



/admin/spaces/:spaceId/blocks y **POST /admin/blocks/:blockId/cancel**) y la exportación del listado de reservas en CSV.

En paralelo a la navegación de la aplicación renderizada con EJS, el proyecto dispone de un cliente React que consume la API REST. En este cliente, el flujo principal parte de la autenticación contra la API y continúa con la consulta de reservas, la creación de nuevas reservas, la edición o cancelación de las existentes y la gestión de reservas recurrentes.

Navegabilidad de la aplicación web renderizada con EJS:

Usuario anónimo

- /login
 - /register
- Registro → correcto → /spaces
- Login → redirección según rol

Usuario estándar

- /spaces
- /spaces/:spaceId
- /spaces/:spaceId/availability
- /reservations/mine
- /account/password
- /account/profile

Usuario administrador

- /admin/reservations
- /admin/spaces
- /admin/spaces/:spaceId/edit
- /admin/spaces/:spaceId/blocks
- /admin/users

Acciones protegidas

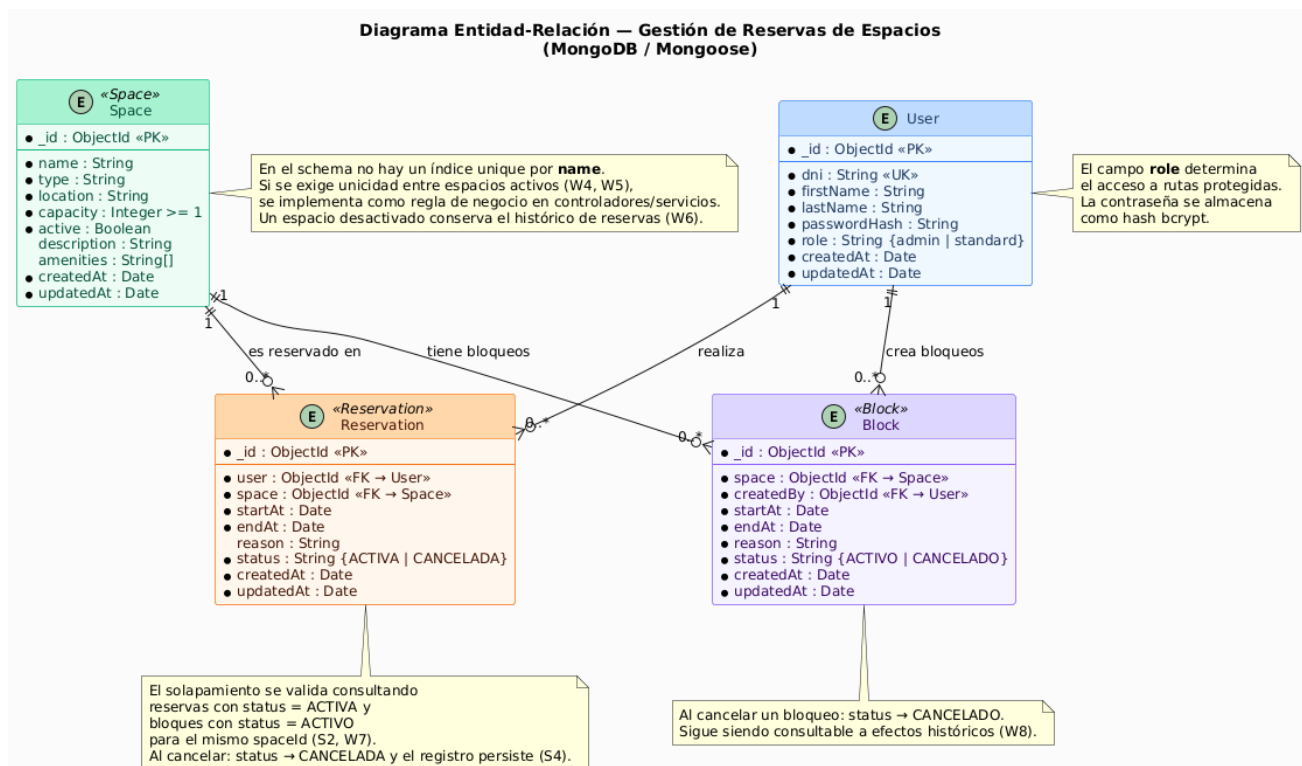
- POST /logout
- POST /reservations/:reservationId/cancel
- POST /admin/spaces (crear)
- POST /admin/spaces/:spaceId/toggle (activar/desactivar)
- POST /admin/spaces/:spaceId/blocks (crear bloqueo)
- POST /admin/blocks/:blockId/cancel (cancelar bloqueo)
- Exportación CSV de reservas

Cliente React

- Login contra API REST
- Consulta de reservas propias

- Creación de reservas
- Edición de reservas
- Cancelación de reservas
- Gestión de reservas recurrentes

3 Diagrama Entidad-Relación



La entidad **User** almacena la información de los usuarios del sistema. Sus campos principales son *dni*, *firstName*, *lastName*, *passwordHash* y *role*, además de los campos de auditoría *createdAt* y *updatedAt*. Cada usuario puede tener asociadas múltiples reservas. En el caso de los usuarios con rol administrador, también pueden aparecer como creadores de bloqueos mediante el campo *createdBy* en la entidad **Block**.

La entidad **Space** representa los espacios que pueden ser reservados. Incluye campos como *name*, *type*, *location*, *capacity*, *description*, *amenities* y *active*, además de *createdAt* y *updatedAt*. Un espacio puede tener múltiples reservas y múltiples bloqueos asociados. El campo *active* permite gestionar la baja lógica del espacio sin eliminar su histórico de reservas.

La entidad **Reservation** modela cada reserva realizada por un usuario sobre un espacio. Sus campos principales son *user*, *space*, *startAt*, *endAt*, *reason* y *status*, junto con los campos de auditoría *createdAt* y *updatedAt*. Cada reserva pertenece a un único usuario y a un único espacio. Los estados definidos son **ACTIVA** y **CANCELADA**, permitiendo mantener el histórico sin eliminar registros.

La entidad **Block** representa los bloqueos administrativos aplicados sobre los espacios, generalmente por tareas de mantenimiento o indisponibilidad temporal. Sus campos son *space*, *createdBy*, *startAt*, *endAt*, *reason* y *status*, además de *createdAt* y *updatedAt*. Cada bloqueo pertenece a un espacio y



queda asociado al usuario administrador que lo creó. Los estados definidos son **ACTIVO** y **CANCELADO**, permitiendo conservar el histórico de bloqueos.

Las relaciones principales del sistema son las siguientes: un usuario puede tener múltiples reservas; un espacio puede tener múltiples reservas; un espacio puede tener múltiples bloqueos; y un usuario con rol administrador puede crear múltiples bloqueos.

La lógica de disponibilidad se basa en la comprobación de solapes temporales. Para determinar si un espacio está libre en un intervalo dado, se verifica la condición de intersección entre intervalos: una reserva o bloqueo existente se considera solapado cuando su *startAt* es anterior al fin solicitado y su *endAt* es posterior al inicio solicitado. Para este cálculo únicamente se tienen en cuenta las reservas con estado **ACTIVA** y los bloqueos con estado **ACTIVO**, pertenecientes al mismo espacio.

Modelo de entidades implementado con Mongoose:

Entidad	Campos principales	Relaciones
User	dni, firstName, lastName, passwordHash, role	Un usuario puede tener muchas reservas. Un administrador puede crear muchos bloqueos.
Space	name, type, location, capacity, description, amenities, active	Un espacio puede tener muchas reservas y muchos bloqueos. La baja lógica se controla con active.
Reservation	user, space, startAt, endAt, reason, status	Pertenece a un usuario y a un espacio. Estados: ACTIVA, CANCELADA.
Block	space, createdBy, startAt, endAt, reason, status	Pertenece a un espacio y es creado por un administrador. Estados: ACTIVO, CANCELADO.

Relaciones principales:

- User 1:N Reservation
- Space 1:N Reservation
- Space 1:N Block
- User (admin) 1:N Block

La disponibilidad se calcula por solape temporal mediante la condición: **startAt < finSolicitado y endAt > inicioSolicitado**, considerando solo reservas ACTIVA y bloqueos ACTIVO.

4 Aspectos técnicos y de diseño relevantes

La aplicación sigue una organización inspirada en el patrón MVC, aplicada de forma práctica y adaptada a las necesidades del proyecto. Las rutas se agrupan en *src/routes*, los controladores en *src/controllers*, los modelos en *src/models*, los servicios en *src/services* y las vistas en *views*. Esta separación facilita la localización de responsabilidades y reduce el acoplamiento entre la lógica de negocio, la persistencia y la presentación.



La persistencia se ha resuelto con MongoDB y Mongoose. Esta decisión permite trabajar con un modelo de datos flexible, adecuado para las entidades del sistema, y simplifica la interacción con la base de datos desde el código de aplicación.

La gestión de sesión en la aplicación web se implementa con express-session y connect-mongo, de modo que las sesiones no quedan en memoria del servidor, sino persistidas en MongoDB. Esto permite mantener la autenticación entre reinicios de la aplicación y evita que el usuario tenga que volver a registrarse o iniciar sesión por una simple parada del servidor.

La autenticación de la aplicación web se apoya en sesiones, mientras que la API REST utiliza autenticación mediante token Bearer. De este modo, se separan correctamente los mecanismos de acceso de la interfaz web tradicional y del cliente React.

Las contraseñas no se almacenan en texto plano, sino mediante hash con bcryptjs, lo que mejora la seguridad del sistema. Además, las validaciones de formato de DNI, robustez de contraseña y otros datos de entrada se centralizan en utilidades compartidas, evitando duplicación de lógica y manteniendo un comportamiento coherente en toda la aplicación.

Uno de los componentes más relevantes de la lógica de negocio es el servicio de disponibilidad, encargado de calcular conflictos entre reservas y bloqueos. Centralizar esta lógica en un servicio específico reduce inconsistencias y facilita su reutilización tanto en la web como en la API.

También resulta relevante el mecanismo de seed de datos iniciales. El proyecto incluye un servicio de carga de datos de prueba que permite reconstruir un estado base cuando se necesite. No obstante, en la configuración actual la aplicación conserva por defecto usuarios, reservas y sesiones entre reinicios, y solo regenera los datos si se activa explícitamente la variable `RESET_DB_ON_START=true`.

En la capa de presentación se emplean vistas EJS y recursos estáticos organizados en public. Esta elección encaja bien con la parte web renderizada en servidor, mientras que el cliente React permite cubrir el apartado de consumo de API y navegación desacoplada en frontend

5 Descripción de funcionalidades desarrolladas por cada miembro del equipo

Miembro del equipo	Funcionalidades desarrolladas
Omayma El Imami Gaada	Omayma se encargó principalmente de todo lo relacionado con la lógica del servidor en el backend utilizando Node.js y Express. En concreto, desarrolló los siguientes requisitos junto con sus correspondientes pruebas: • Requisito W4: Creación de espacios en el sistema. • Requisito W5: Edición de espacios. • Requisito W6: Activación y desactivación de espacios. • Requisito W7: Creación de bloqueos de espacios.



	• Requisito W8: Cancelación de bloqueos. • Requisito W9: Consulta del listado global de reservas. • Requisito W13: Implementación de la seguridad del sistema, incluyendo la protección de rutas y acciones mediante control de acceso. • Requisito W15: Consulta del listado de usuarios. • Requisito W16: Exportación de información en formato CSV. Además, asumió como responsabilidad adicional la implementación del middleware de autenticación y el control de roles (administrador/usuario), asegurando que cada funcionalidad estuviera restringida al tipo de usuario correspondiente. Finalmente, tras la finalización del desarrollo principal, participó en la elaboración del diagrama de navegabilidad y del diagrama entidad-relación, así como en la revisión de la aplicación, implementando las pruebas correspondientes a su parte y corrigiendo errores detectados durante el desarrollo. Además, también colaboró en la redacción final del documento en Word.
Ikram El Mabrouk Morhnane	Ikram se encargó principalmente de la parte relacionada con la interfaz web, las vistas y la experiencia de usuario dentro del sistema. En concreto, desarrolló los siguientes requisitos junto con sus correspondientes pruebas: • Requisito W1: Registro de usuario en la aplicación. • Requisito W2: Inicio de sesión de usuarios. • Requisito W3: Cierre de sesión. • Requisito W10: Visualización del listado de espacios. • Requisito W11: Consulta del detalle de un espacio.



	• Requisito W12: Comprobación de la disponibilidad de un espacio. • Requisito W14: Modificación de la contraseña del usuario. • Además, asumió como responsabilidad adicional el diseño de la interfaz (UX/UI), asegurando que la aplicación fuera visualmente clara y fácil de usar, así como la implementación de validaciones en el frontend para mejorar la interacción del usuario. Asimismo, tras la finalización del desarrollo principal, participó en la explicación de los diagramas incluidos en el documento, colaboró en la revisión de la aplicación implementando las pruebas correspondientes a su parte y corrigiendo errores detectados tras la integración del sistema, y también contribuyó a la redacción final del informe en Word.
Moisés Montes Iglesias	Moisés se encargó principalmente del desarrollo del cliente en React, la integración con la API REST y la parte de testing. En concreto, desarrolló los siguientes requisitos junto con sus correspondientes pruebas: • Requisito C1: Inicio de sesión en el cliente React. • Requisito C2: Creación de reservas. • Requisito C3: Consulta de reservas. • Requisito C4: Cancelación de reservas. • Requisito C5: Edición de reservas. • Requisito C6: Gestión de reservas recurrentes. • Además, asumió como responsabilidad adicional la integración completa del cliente con la API, gestionando el estado de la aplicación y asegurando la correcta sincronización de los datos entre frontend y backend. También se encargó de la realización de pruebas funcionales utilizando Selenium y JUnit, comprobando el correcto funcionamiento de las distintas funcionalidades del sistema.



	Asimismo, participó en la resolución de errores detectados durante la integración del proyecto, contribuyendo a mejorar la estabilidad de la aplicación. Finalmente, colaboró en la redacción del documento final (Word).
Mario de la Mora Martínez	Mario se encargó principalmente del desarrollo de la API REST y de la seguridad mediante autenticación con token. En concreto, desarrolló los siguientes requisitos junto con sus correspondientes pruebas: • Requisito S1: Login mediante token. • Requisito S2: Creación de reservas. • Requisito S3: Consulta de reservas propias. • Requisito S4: Cancelación de reservas. • Requisito S5: Edición de reservas. • Requisito S6: Gestión de reservas recurrentes. • Además, asumió como responsabilidad adicional la correcta gestión de códigos HTTP, la estructuración de respuestas en formato JSON y la implementación de validaciones en el backend, asegurando una comunicación adecuada entre cliente y servidor. Asimismo, participó en la resolución de errores tanto en el código como en algunos tests, especialmente aquellos derivados de la integración de los distintos commits del equipo, y en la revisión de la internacionalización de la aplicación. Finalmente, colaboró en la redacción del documento final (Word).



Commits de los integrantes:

Se debe tener en cuenta que, a diferencia del proyecto anterior, en este caso el equipo ha optado por una forma de trabajo más centralizada, realizando en muchas ocasiones el desarrollo sobre una misma rama o repositorio compartido. Esta decisión se tomó con el objetivo de optimizar el tiempo disponible, ya que coincidía con la carga de otras asignaturas. Por este motivo, el número de commits individuales puede ser inferior por cada integrante.

★ Commits de Omayma El Imami Gaada:

Commits

main

uo300838 All time

Commits on May 3, 2026

- Merge pull request #15 from Ikram-em/uo300838-rama Verified ae79f85
- proyecto con los tests dentro en el sitio correcto uo300838 committed 25 minutes ago 61b17b7
- Merge pull request #14 from Ikram-em/uo300838-rama Verified 65e102d
- cambios para que pasen pruebas y cambio de una interfaz uo300838 committed 1 hour ago 9e2adea
- arreglo de tests y otras cosas uo300838 committed 12 hours ago a15d090

Commits on May 1, 2026

- Fix Selenium WebFrontendSeleniumTests uo300838 committed 3 days ago 0af1207

Commits on Apr 28, 2026

- Merge pull request #3 from Ikram-em/feature/backend-omayma Verified 48767a9
- feat: add mongodb backend for admin web flows uo300838 committed last week 94c6d37

★ Commits de Ikram El Mabrouk Morhnane:

Commits

main

Ikram-em All time

Commits on May 1, 2026

- Merge pull request #10 from Ikram-em/uo300838-rama Verified 09a8f32

Commits on Apr 30, 2026

- Merge pull request #9 from Ikram-em/Interfaces Verified b50c315
- Polish UI and persistence flow Ikram-em committed 15 hours ago 8db0ba4

Commits on Apr 27, 2026

- Merge pull request #1 from Ikram-em/feature/jquery-frontend-ikram Verified 655f6f4
- feat: add jquery frontend for auth and spaces Ikram-em committed 4 days ago 16e5b1f
- chore: initialize repository Ikram-em committed 4 days ago 40973ba



★ Commits de Moisés Montes Iglesias:

Commits		
main	MoisesMontesIgl...	All time
Commits on Apr 29, 2026		
Merge pull request #8 from Ikram-em/feature/REACT-moisés	Verified 5af0634	
MoisesMontesIglesias authored 2 days ago		
Tests terminados de REACT	036fede	
MoisesMontesIglesias committed 2 days ago		
Tests finalizados de REACT	0751405	
MoisesMontesIglesias committed 2 days ago		
Merge pull request #7 from Ikram-em/feature/REACT-moisés	Verified 823d0a4	
MoisesMontesIglesias authored 2 days ago		
Pruebas de Selenium completas	3c6df4d	
MoisesMontesIglesias committed 2 days ago		
Merge pull request #6 from Ikram-em/feature/REACT-moisés	Verified 3ecb1a6	
MoisesMontesIglesias authored 2 days ago		
Ejercicio de React C6 con pruebas 59, 60	2a1f105	
MoisesMontesIglesias committed 2 days ago		
Ejercicio de React C5 con pruebas 57, 58	f390afa	
MoisesMontesIglesias committed 2 days ago		
Ejercicio de React C4 con prueba 56	0f3a080	
MoisesMontesIglesias committed 2 days ago		
Ejercicio de React C3 con pruebas 54, 55	b2b134e	
MoisesMontesIglesias committed 2 days ago		
Ejercicio de React C2 con pruebas 52,53	816eebd	
MoisesMontesIglesias committed 2 days ago		
Ejercicio de React C1 con pruebas 49,50,51	6461bba	
MoisesMontesIglesias committed 2 days ago		

★ Commits de Mario de la Mora Martínez:

Commits on May 3, 2026		
Merge pull request #12 from Ikram-em/Mario	Verified 1cf26ef	
uo281860 authored 14 hours ago		
Comentarios Swagger api rest	256c96c	
uo281860 committed 14 hours ago		
Commits on May 2, 2026		
Merge pull request #11 from Ikram-em/Mario	Verified 999cf92	
uo281860 authored yesterday		
Test Selenium Api REST	59c3895	
uo281860 committed yesterday		
Commits on Apr 29, 2026		
Merge pull request #5 from Ikram-em/Mario	Verified a0c4f36	
uo281860 authored 4 days ago		
Add REST API files	c6581df	
uo281860 committed 4 days ago		
Commits on Apr 28, 2026		
Merge pull request #4 from Ikram-em/Mario	Verified 6c5a2fd	
uo281860 authored 5 days ago		
Api rest	d64a6b9	
uo281860 committed 5 days ago		



6 Conclusión individual de cada miembro del grupo.

Cada integrante del equipo ha participado en una parte distinta del proyecto, lo que ha permitido repartir responsabilidades y avanzar en paralelo sobre distintas capas de la aplicación.

- En mi caso (Ikram), lo más clave para mí fue la buena coordinación que tuve con mis compañeros, hemos trabajado de forma colaborativa, ayudándonos entre nosotros cuando surgían dudas o problemas. En cuanto a la implementación, me costó un poco más la parte de manejar la coherencia entre la interfaz, la navegación y los permisos de usuario, ya que era importante que cada vista mostrara la información adecuada según el rol. Además, durante el desarrollo tuve algunos problemas con el entorno de trabajo, ya que en varias ocasiones se producían fallos con las dependencias del proyecto (node_modules), lo que me obligaba a reinstalarlas o reconfigurar el entorno para poder continuar trabajando.
- En mi caso (Omayma,) concuerdo con mi compañera Ikram en cuanto a la buena coordinación de mis compañeros. En la implementación, una de las partes en la que más he notado que iba lento fue a la hora de gestionar los espacios, bloqueos y reservas, así como la seguridad mediante middleware de autenticación, ya que quería garantizar que todo funcionaba correctamente y para ello iba probando cada cosa.
- En mi caso (Mario), trabajar en grupo ha sido clave para poder sacar adelante el proyecto, ya que, aunque el tiempo era bastante ajustado, la buena coordinación y la ayuda entre compañeros han facilitado mucho el desarrollo. En cuanto a la implementación, me he centrado en el desarrollo de la API REST y la seguridad mediante token. Esta parte ha requerido bastante atención, ya que era fundamental que la comunicación entre cliente y servidor funcionara correctamente. Al principio me costó un poco más adaptarme, sobre todo a la hora de diseñar los endpoints y asegurar que las respuestas fueran coherentes. Además, también tuve que prestar especial atención a las validaciones en backend y a funcionalidades como el login, la creación, consulta y cancelación de reservas.
- En mi caso (Moisés), el trabajo en grupo ha sido muy importante para poder integrar correctamente todas las partes del sistema, ya que cada uno se encargaba de una parte distinta. La comunicación entre nosotros ha ayudado bastante a resolver problemas y avanzar más rápido. En cuanto a la implementación, una de las partes que más me ha costado ha sido gestionar el sincronizar correctamente los datos con la API. También ha sido importante la realización de pruebas funcionales con Selenium y JUnit, ya que había que comprobar que todas las funcionalidades funcionaban correctamente.



7 Información necesaria para el despliegue y ejecución

Para ejecutar correctamente el proyecto es necesario disponer de un entorno con Node.js, npm y MongoDB en local. La base de datos se configura para trabajar por defecto contra **mongodb://127.0.0.1:27017/sdi2526-entregable2-72**.

- La instalación de dependencias se realiza con: **npm install**
- El arranque de la aplicación principal se realiza con: **npm start**
- Una vez iniciada, la aplicación queda disponible por defecto en: **http://localhost:3000**

La configuración base puede ajustarse a través de un fichero `.env`, tomando como referencia `.env.example`. Entre las variables más relevantes se encuentran:

- **MONGODB_URI**: URI de conexión a MongoDB.
- **SESSION_SECRET**: clave de firma de sesión.
- **RESET_DB_ON_START**: si vale `true`, regenera los datos de prueba al arrancar; si vale `false`, conserva los datos existentes.

La aplicación incorpora un usuario administrador de prueba con las siguientes credenciales:

- DNI: 12345678Z
- Contraseña: @Dm1n1str@D0r

- La API REST está disponible bajo el prefijo `/api` y utiliza autenticación mediante **Bearer token**.
- El cliente React puede ejecutarse en modo desarrollo con: **npm run react:dev**
- Y generarse para producción con: **npm run react:build**
- El comando: **npm test**, no ejecuta pruebas Node directamente, sino que redirige al módulo de pruebas ubicado en `sdi2526-entrega2-test-72`, donde se encuentran las pruebas funcionales desarrolladas con **Selenium y JUnit**.

- Repositorio: [haz click aqui](#)

8 Conclusión

El proyecto desarrollado constituye una solución completa para la gestión de reservas de espacios, con una separación clara entre la interfaz web, la lógica de negocio, la persistencia, la API REST y el cliente React.

A lo largo del desarrollo, el trabajo en grupo ha sido un aspecto clave, ya que hemos podido organizarnos bien, repartir tareas y ayudarnos entre nosotros cuando surgían problemas. Esto ha permitido avanzar de forma más eficiente y entender mejor las distintas partes del sistema, no solo la propia.

La aplicación cubre los flujos esenciales de autenticación, consulta de espacios, control de disponibilidad, gestión de reservas y administración del sistema. Además, la arquitectura adoptada



facilita la mantenibilidad del código y permite que el proyecto pueda ampliarse en el futuro sin grandes dificultades.

Uno de los aspectos más importantes ha sido la correcta coordinación entre las distintas partes desarrolladas por cada miembro del equipo, así como la integración final entre frontend, backend y API.

En conjunto, el resultado final cumple con los objetivos planteados y refleja tanto el trabajo técnico realizado como la buena organización y colaboración del grupo.